Milestone 5: Mutation Testing and Test Suite Quality

CMPT 470/816: Advanced Software Engineering

University of Saskatchewan

Contents

1 Overview 3

2 Before You Start: Project Compatibility Checklist 3

3 Prerequisites 4

4 Required Software 4

5 Setting Java 17 for Your Terminal Session 5

6 Clone and Verify the Build 5

7 Step 1 — Clone the Repository 5

8 Step 2 — Confirm the Project Compiles 5

9 Step 3 — Confirm All Tests Pass 6

10 Finding Your Base Package Name 6

11 Adding PIT to `pom.xml` 6

12 Test Framework Configuration 6

13 The Plugin Block 7
- 13.1 JUnit 5 (or Mixed JUnit 4 + 5) . . . 7
- 13.2 JUnit 4 Only . . . 9

14 Fields to Customise 10

15 Reducing Mutation Scope 10

16 Running PIT 11

17 The Run Command 11

18 What a Healthy Run Looks Like 11

19 Understanding the Final Statistics 12

20 Expected Runtime by Scope 12

21 Reading the HTML Report 12

22 Finding the Report 12

23 Navigating the Report 13

24 Finding Surviving Mutants from the Terminal 13

25 Common Variations Across Repositories 13

26 Multi-Module Projects 14

27 Early Progress Checkpoint 15

28 Error Reference 16

29 E1 — TestEngine `junit-jupiter` Failed: Java 8 Is Active 16

30 E2 — TestEngine `junit-jupiter` Failed: Plugin in Wrong Location 16

31 E3 — `OutputDirectoryCreator` Not Available: Version Mismatch 17

32 E4 — Coverage Minion Exited `UNKNOWN_ERROR`: Network Tests 17

33 E5 — `ClassNotFoundException` in Minion: Multi-Release JAR 18

34 E6 — *N* Tests Did Not Pass Without Mutation 18

35 E7 — Many `TIMED_OUT` Mutants in Results 18

36 E8 — `BUILD SUCCESS` but Zero Mutations Generated 19

37 Quick-Reference Card 20

38 Essential Commands 20

39 Pre-Flight Checklist 20

40 Error Quick-Lookup 21

1 Overview

This guide explains how to configure and run **PIT** (Pitest), a bytecode-level mutation testing tool for Java Maven projects using JUnit 5. It covers the full workflow from prerequisites to reading the final report, and includes a structured error reference for every known failure mode with exact fixes derived from real debugging sessions.

What PIT does. PIT systematically alters your compiled production bytecode—one small change at a time—producing modified copies called *mutants*. It then re-runs your test suite against each mutant. A mutant is *killed* if at least one test fails on it; it *survives* if all tests still pass. Surviving mutants indicate gaps in test quality that line coverage alone cannot reveal.

Scope of this guide. These instructions apply to Maven projects using JUnit 4 or JUnit 5, including multi-module layouts. All customisation points are explicitly labelled; repository-specific variations are addressed in Section 25.

2 Before You Start: Project Compatibility Checklist

Complete every check below **before** touching `pom.xml`. Skipping this step is the most common cause of wasted debugging time.

1 — The project builds with Maven

```
mvn clean install -DskipTests
```

Listing 1: Verify the project builds before adding PIT

Expected result

`BUILD SUCCESS`. If this fails, fix the build first. PIT cannot run on a project that does not compile.

2 — Maven is the build system

Check that a `pom.xml` file exists in the directory you are working in:

```
ls pom.xml    # must exist
```

Listing 2: Confirm pom.xml is present

Warning

If your project uses Gradle (`build.gradle` instead of `pom.xml`), the Maven plugin block in this guide does not apply. Use the Gradle PIT plugin[a] instead.

[a] https://gradle-pitest-plugin.solidsoft.info/

3 — Identify the test framework

The JUnit version determines whether you need the `pitest-junit5-plugin` bridge (see Section 12).

```
# Check which JUnit dependency is declared:
grep -i "junit" pom.xml

# JUnit 5 signature:
#   <artifactId>junit-jupiter</artifactId>
#   or <artifactId>junit-jupiter-api</artifactId>

# JUnit 4 signature:
#   <artifactId>junit</artifactId>   (groupId: junit)
```

Listing 3: Identifying the JUnit version from pom.xml

4 — Identify the project structure

```
ls -d */   # list subdirectories

# Single-module: only src/ and target/ appear -> continue normally
# Multi-module:  subdirectories with their own pom.xml appear
#                -> navigate into the specific module before running PIT
#                   (see Section~\ref{sec:multimodule})
```

Listing 4: Detecting single-module vs multi-module layout

Note

If **any** of the four checks above reveal a problem, resolve it before proceeding. The remaining sections assume all four checks pass.

3 Prerequisites

4 Required Software

Tool	Required version	Verify with
Java JDK	**17 or Higher**	`java -version`
Maven	3.6.3 or newer	`mvn -version`
Git	Any recent version	`git --version`
RAM	8 GB recommended	—
Disk	∼2 GB free	`df -h .`

Table 1: Required software and verification commands

5 Setting Java 17 for Your Terminal Session

If your system defaults to an older Java version, override it for the current terminal session before running any Maven commands.

```
# Linux
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
mvn -version          # must show: Java version: 17.x.x

# macOS (Homebrew)
export JAVA_HOME=/opt/homebrew/opt/openjdk@17
export PATH=$JAVA_HOME/bin:$PATH
mvn -version
```

Listing 5: Switching to Java 17 in the current terminal session

Note

These exports last only for the current terminal session. To make them permanent, add them to `~/.bashrc` or `~/.zshrc`.

6 Clone and Verify the Build

Before adding PIT, confirm the project builds cleanly and all tests pass. PIT refuses to run against a red test suite.

7 Step 1 — Clone the Repository

```
git clone <your-repo-url>
cd <repo-directory>
```

Listing 6: Clone the repository

8 Step 2 — Confirm the Project Compiles

```
mvn clean compile -q
```

Listing 7: Compile the project

Expected result

`BUILD SUCCESS` with no errors. Fix all compile errors before proceeding.

9 Step 3 — Confirm All Tests Pass

```
mvn clean test
```

Listing 8: Run the full test suite

Expected result

`BUILD SUCCESS` — 0 failures, 0 errors. This is a **hard requirement** before running PIT.

Warning

PIT will refuse to run if even one test fails. It prints:
`N tests did not pass without mutation when calculating line coverage.`
Fix all test failures before proceeding.

10 Finding Your Base Package Name

You need the base package name for the `targetClasses` and `targetTests` settings in the next section.

```
# List top-level directories in compiled output:
ls target/classes/

# Typical results:
#   com/   ->  use  com.yourproject.*
#   org/   ->  use  org.yourproject.*
#   io/    ->  use  io.yourproject.*

# Confirm with a deeper look:
find target/classes -name '*.class' | head -5
```

Listing 9: Identifying the base package name from compiled output

11 Adding PIT to `pom.xml`

Open `pom.xml` in the repository root (or module root for multi-module projects). Locate the `<build><plugins>` section and insert the plugin block below just before the closing `</plugins>` tag.

12 Test Framework Configuration

PIT's dependency block differs depending on which test framework the project uses. Identify your framework using the check in Section 2 and follow the matching instructions.

JUnit 5 (junit-jupiter). You **must** include the `pitest-junit5-plugin` bridge as a nested

dependency inside the plugin block. Without it PIT reports 0 tests and silently produces no mutations.

JUnit 4 (junit 4.x). Do **not** include the `pitest-junit5-plugin`. PIT has built-in JUnit 4 support; adding the bridge causes duplicate `TestEngine` registration and breaks test discovery.

Mixed JUnit 4 + JUnit 5. Include the `pitest-junit5-plugin` but also verify that your `pitest-maven` version is at least `1.15.0`, which handles dual-engine classpath detection automatically. Additional Surefire configuration may be required.

Framework	Plugin dependency needed?	Note
JUnit 5 only	**Yes** — `pitest-junit5-plugin:1.2.2`	Must be inside plugin block, NOT top-level
JUnit 4 only	**No**	Built-in; adding the bridge breaks discovery
Mixed 4 + 5	**Yes** — same as JUnit 5	Requires `pitest-maven` $\geq$ 1.15.0

Table 2: JUnit framework vs. pitest-junit5-plugin requirement

13 The Plugin Block

The PIT plugin configuration depends on your test framework (see Section 12).

Select the appropriate version below and modify only the items marked in comments (e.g., target package, threads).

13.1 JUnit 5 (or Mixed JUnit 4 + 5)

Use this configuration if your project uses JUnit 5 or a mix of JUnit 4 and JUnit 5.

PIT plugin block (JUnit 5 / Mixed)

```
<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>1.15.3</version>

  <dependencies>
    <!-- REQUIRED for JUnit 5 -->
    <dependency>
      <groupId>org.pitest</groupId>
      <artifactId>pitest-junit5-plugin</artifactId>
      <version>1.2.2</version>
    </dependency>
  </dependencies>

  <configuration>

    <!-- Change to your repo's base package -->
```

```
    <targetClasses>
      <param>org.yourpackage.*</param>
    </targetClasses>

    <!-- Same base package for tests -->
    <targetTests>
      <param>org.yourpackage.*</param>
    </targetTests>

    <!-- Optional: exclude slow or integration tests -->
    <!--
    <excludedTestClasses>
      <param>org.yourpackage.integration.*</param>
    </excludedTestClasses>
    -->

    <mutators>
      <mutator>DEFAULTS</mutator>
    </mutators>

    <outputFormats>
      <param>HTML</param>
      <param>XML</param>
    </outputFormats>

    <!-- Adjust based on your machine -->
    <threads>3</threads>

    <!-- Increase if many mutants time out -->
    <timeoutConstant>5000</timeoutConstant>

    <!-- Increase memory if PIT crashes -->
    <jvmArgs>
      <value>-Xmx512m</value>
    </jvmArgs>

    <verbose>true</verbose>
    <withHistory>false</withHistory>

  </configuration>
</plugin>
```

Warning

Only for JUnit 5 or mixed projects:
Do **not** add `pitest-junit5-plugin` as a top-level `<dependency>`. It must appear only inside the PIT plugin block.
Placing it at the top level can break test discovery.

13.2 JUnit 4 Only

Use this configuration if your project uses only JUnit 4.

PIT plugin block (JUnit 4)

```
<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>1.15.3</version>

  <!-- No pitest-junit5-plugin here -->

  <configuration>

    <!-- Change to your repo's base package -->
    <targetClasses>
      <param>org.yourpackage.*</param>
    </targetClasses>

    <!-- Same base package for tests -->
    <targetTests>
      <param>org.yourpackage.*</param>
    </targetTests>

    <!-- Optional: exclude slow or integration tests -->
    <!--
    <excludedTestClasses>
      <param>org.yourpackage.integration.*</param>
    </excludedTestClasses>
    -->

    <mutators>
      <mutator>DEFAULTS</mutator>
    </mutators>

    <outputFormats>
      <param>HTML</param>
      <param>XML</param>
    </outputFormats>

    <!-- Adjust based on your machine -->
    <threads>3</threads>

    <!-- Increase if many mutants time out -->
    <timeoutConstant>5000</timeoutConstant>

    <!-- Increase memory if PIT crashes -->
    <jvmArgs>
      <value>-Xmx512m</value>
    </jvmArgs>

    <verbose>true</verbose>
```

```
    <withHistory>false</withHistory>

  </configuration>
</plugin>
```

14 Fields to Customise

Field	What to set	Example
`targetClasses`	Your repo's root package	`com.google.gson.*`
`targetTests`	Same root package	`com.google.gson.*`
`threads`	CPU cores minus one	4 (5-core machine)

Table 3: Settings that must be customised per project

15 Reducing Mutation Scope

By default PIT mutates every class matched by `targetClasses`. On large repositories this can generate thousands of mutants and run for over an hour. The following options reduce scope without changing the core configuration.

Option A — Target a single sub-package. Keep `targetTests` broad so all tests can still cover it.

```
<targetClasses>
  <param>org.yourpackage.parser.*</param>  <!-- one sub-package -->
</targetClasses>
<targetTests>
  <param>org.yourpackage.*</param>         <!-- all tests -->
</targetTests>
```

Listing 10: Narrowing scope to a single sub-package

Option B — Target a single class. Useful when investigating a specific surviving mutant for Experiment 3.

```
<targetClasses>
  <param>org.yourpackage.parser.SomeParser</param>
</targetClasses>
```

Listing 11: Targeting a single class

Option C — Cap mutants per class. Limits to *N* mutants per class across the whole scope.

```
<features>
  <feature>+CLASSLIMIT(limit[25])</feature>
</features>
```

Listing 12: Capping mutations per class

Option D — Fewer mutation operators. Reduces mutant count by approximately 70% compared to `DEFAULTS`.

```
<mutators>
  <mutator>CONDITIONALS_BOUNDARY</mutator>
  <mutator>NEGATE_CONDITIONALS</mutator>
</mutators>
```

Listing 13: Using a reduced set of mutation operators

16 Running PIT

17 The Run Command

Always pipe output through `tee` so the log is saved. If the run fails, the log is the primary diagnostic tool.

```
mvn org.pitest:pitest-maven:mutationCoverage 2>&1 | tee pit_run.log
```

Listing 14: Running PIT and saving the log simultaneously

18 What a Healthy Run Looks Like

With `verbose=true` the following lines appear in order. Each one confirms that a stage of the run has completed successfully.

```
[INFO] Found shared classpath plugin : JUnit 5 test framework support
[INFO] Created N mutation test units in pre scan
[INFO] Sending N test classes to minion
MINION: Checking environment
MINION: Gathering coverage for test Description [testClass=...]
MINION: mutating method someMethod
MINION: N relevant tests for someMethod
...
>> Line Coverage: XXXX/YYYY (ZZ%)
>> Generated NNNN mutations Killed MMMM (XX%)
>> Mutations with no coverage NNN. Test strength ZZ%
[INFO] BUILD SUCCESS
```

Listing 15: Console output of a successful PIT run (with verbose=true)

Note

High fan noise and sustained CPU usage during the run are normal. PIT forks a new JVM process per mutation batch, using all configured threads in parallel. Do not kill the process.

19 Understanding the Final Statistics

Output line	Meaning
`Generated N mutations Killed M (X%)`	**Overall mutation score.** Killed / all mutants including those with no test coverage. This is the headline number to report.
`Test strength ZZ%`	Killed / mutants that tests actually reached. Always higher than the mutation score. Measures test quality independently of coverage gaps.
`Mutations with no coverage N`	Mutants in code that no test ever executes. They count against the mutation score but not against test strength.
`Line Coverage: XXXX/YYYY`	Standard line coverage. Does not reflect mutation quality — only whether lines were executed at all.

Table 4: Meaning of PIT's final output statistics

20 Expected Runtime by Scope

Approx. mutants	threads=2	threads=3	threads=4
< 500	5–10 min	3–6 min	2–4 min
500–2 000	15–30 min	10–20 min	7–14 min
2 000–4 000	35–60 min	22–40 min	15–28 min
$> 4\,000$	> 60 min	$\approx$30 min	$\approx$20 min

Table 5: Approximate runtimes by mutation count and thread count

21 Reading the HTML Report

22 Finding the Report

After `BUILD SUCCESS`, the report is written to:

```
target/pit-reports/index.html

# Open in a browser:
xdg-open target/pit-reports/index.html    # Linux
open target/pit-reports/index.html        # macOS
```

Listing 16: Opening the PIT HTML report

23 Navigating the Report

The **index page** shows overall mutation score, test strength, and a sortable table of all mutated classes with their individual scores.

Clicking any class name opens the **line-by-line mutation view**. Line background colours indicate outcomes:

- **Green** — at least one mutant on this line was killed by a test.
- **Red** — a mutant **survived**: no test detected the behavioural change. These are the primary targets for Experiment 3.
- **Yellow** — no test reached this line (no coverage).

Hovering over a highlighted line shows the exact mutation operator applied (e.g. `CONDITIONALS_BOUNDARY`, `NEGATE_CONDITIONALS`, `VOID_METHOD_CALLS`).

24 Finding Surviving Mutants from the Terminal

The XML report provides structured access to every mutant's details.

```
# List all surviving mutants (class, method, line, operator):
grep -A5 'status="SURVIVED"' target/pit-reports/mutations.xml | head -60

# Count survivors per class:
grep 'status="SURVIVED"' target/pit-reports/mutations.xml | \
  grep -o 'mutatedClass="[^"]*"' | sort | uniq -c | sort -rn
```

Listing 17: Querying surviving mutants from the XML report

Note

`target/pit-reports/mutations.xml` records every mutant with its class name, method name, line number, mutation operator, and final status. It is the most reliable way to enumerate survivors for Experiment 3 without manually clicking through the HTML report.

25 Common Variations Across Repositories

The configuration in Section 11 is a general baseline. Different repositories behave differently and may require adjustments. The table below maps common project characteristics to the

parameters most likely to need changing.

Characteristic	What may happen	Adjust
Gradle build	Maven plugin block does not apply	Use Gradle PIT plugin
Multi-module layout	PIT runs in wrong directory or finds no classes	`cd` into module first (Section 26)
JUnit 4 only	Adding junit5-plugin breaks test discovery	Omit `pitest-junit5-plugin` entirely
Slow or large test suite	Many `TIMED_OUT`; run takes >1 hour	`timeoutConstant`, `threads`, scope
External dependencies (DB, API)	Integration tests crash minion	`excludedTestClasses`
Multi-release JAR	`ClassNotFoundException` in minion	`excludedTestClasses` (see E5)

Table 6: Common repository variations and the parameters they affect

Note

You are not expected to match every repository quirk up front. Start with the baseline config, run PIT once with `verbose=true`, and adjust based on what the log tells you. The Error Reference in Section 28 maps every known symptom to its fix.

26 Multi-Module Projects

If your repository contains multiple Maven modules (subdirectories each with their own `pom.xml`), do **not** run PIT from the repository root. Running from the root causes PIT to find no classes or to attempt mutation across modules it cannot instrument.

Step 1 — Navigate into the target module

```
# Example: working inside the 'core' module
cd core

# Confirm this module has its own pom.xml and source tree:
ls pom.xml src/
```

Listing 18: Navigating into a specific module

Step 2 — Verify the module builds and tests pass

```
mvn clean test
```

Listing 19: Confirm the module's test suite is green before running PIT

Step 3 — Run PIT from inside the module

```
mvn org.pitest:pitest-maven:mutationCoverage 2>&1 | tee pit_run.log
```

Listing 20: Run PIT from within the target module directory

Note

Set `targetClasses` to the module's own base package only. Using a top-level wildcard like `com.example.*` from inside a module will attempt to mutate classes from sibling modules that are not on the module's compile classpath, causing `ClassNotFoundException`.

27 Early Progress Checkpoint

Warning

Do not leave PIT setup to the last minute. Debugging configuration issues typically takes 30–90 minutes even with this guide. Build or test suite problems can add hours.

Within the **first 2–3 days** of the milestone, complete the following three items and confirm each one succeeds:

1. **Project builds successfully**
   `mvn clean install -DskipTests` exits with `BUILD SUCCESS`.
2. **Test suite is green**
   `mvn clean test` exits with `BUILD SUCCESS`, 0 failures, 0 errors.
3. **PIT configuration is in place and produces output**
   Add the plugin block, run `mvn org.pitest:pitest-maven:mutationCoverage` on a **single class or small package**, and confirm a report is generated at `target/pit-reports/index.html`.

If any of these three items has not succeeded by day 3, contact your TA immediately with the full terminal output. Do not wait until the deadline.

28 Error Reference

This section documents every failure mode encountered during PIT setup, with verbatim symptoms, root causes, and fixes. Work through errors in the order they appear: later errors sometimes mask earlier ones.

29 E1 — TestEngine `junit-jupiter` Failed: Java 8 Is Active

E1 — Symptom

```
Tests run: 0
MINION: TestEngine with ID 'junit-jupiter' failed to discover tests
```

Cause. Maven is running under Java 8. JUnit Jupiter 5.x requires Java 11 or higher. The PIT coverage minion cannot load any test classes under Java 8.

Fix.

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
mvn -version    # must show Java version: 17.x.x
```

Listing 21: E1 fix: activate Java 17

30 E2 — TestEngine `junit-jupiter` Failed: Plugin in Wrong Location

E2 — Symptom

Same `TestEngine with ID 'junit-jupiter' failed to discover tests` error as E1, but `mvn -version` already confirms Java 17 is active.

Cause. `pitest-junit5-plugin` was added as a top-level project `<dependency>` instead of inside the plugin block. This places it on Surefire's forked JVM classpath, registering a duplicate `TestEngine` and corrupting JUnit Platform's `ServiceLoader`.

Fix. Remove `pitest-junit5-plugin` from the top-level `<dependencies>` section entirely. It must only appear as a nested `<dependency>` inside the `pitest-maven` plugin block, exactly as shown in Listing 4.1.

31 E3 — `OutputDirectoryCreator` Not Available: Version Mismatch

E3 — Symptom

```
Caused by: JUnitException: OutputDirectoryCreator not available;
probably due to unaligned versions of junit-platform-engine and
junit-platform-launcher jars on the classpath/module path.
```

Cause. `pitest-junit5-plugin:1.2.1` bundles `junit-platform-launcher:1.9.2`. Projects using JUnit 5.11+ require JUnit Platform 1.11+, which introduced the `OutputDirectoryCreator` API. The two versions conflict on the classpath visible to PIT's minion.

Fix. Upgrade `pitest-junit5-plugin` from `1.2.1` to `1.2.2`.

```
<artifactId>pitest-junit5-plugin</artifactId>
<version>1.2.2</version>   <!-- was 1.2.1 -->
```

Listing 22: E3 fix: upgrade pitest-junit5-plugin to 1.2.2

32 E4 — Coverage Minion Exited `UNKNOWN_ERROR`: Network Tests

E4 — Symptom

```
Coverage generator Minion exited abnormally due to UNKNOWN_ERROR
MINION: java.net.UnknownHostException: system.proxy
  (or: java.net.BindException: Address already in use)
```

Cause. Test classes that start an embedded HTTP server (e.g. Jetty, MockWebServer) or make real network calls fail inside PIT's isolated minion process, which runs without network access. The minion JVM crashes during the coverage collection phase before any mutations are attempted.

Fix. With `verbose=true` enabled, identify the failing tests from `SEVERE` log lines:

```
SEVERE: Description [testClass=com.example.IntegrationTest, ...]
        did not pass without mutation.
```

Listing 23: E4 diagnosis: locating network-dependent tests in verbose output

Add each offending class to `<excludedTestClasses>` in the PIT configuration:

```
<excludedTestClasses>
  <param>com.example.integration.*</param>
  <param>com.example.SomeNetworkTest</param>
</excludedTestClasses>
```

Listing 24: E4 fix: exclude network-dependent test classes

33 E5 — `ClassNotFoundException` in Minion: Multi-Release JAR

E5 — Symptom

```
MINION: IllegalStateException: Could not load com.example.SomeClass
MINION: Caused by: ClassNotFoundException: com.example.SomeClass
```

Cause. Some projects compile certain classes only for Java 11+ and store them under `META-INF/versions/11/` (the multi-release JAR format). PIT's coverage minion reads `target/classes` as a flat directory and never sees those versioned class files. Any test that references them fails to load and crashes the minion.

Fix. Exclude the test classes that reference the unloadable multi-release classes:

```
<excludedTestClasses>
  <param>com.example.MultiReleaseFeatureTest</param>
</excludedTestClasses>
```

Listing 25: E5 fix: exclude tests referencing multi-release classes

34 E6 — *N* Tests Did Not Pass Without Mutation

E6 — Symptom

```
Failed to execute goal pitest-maven:mutationCoverage:
N tests did not pass without mutation when calculating line coverage.
Mutation testing requires a green suite.
```

Cause. PIT runs every targeted test once in unmutated code (the baseline pass) before introducing any mutations. Tests that fail here are usually ones that require network access, file-system resources, or services unavailable inside the minion's isolated JVM process.

Fix. If `verbose=true` is not yet set, add it and re-run. Find the `SEVERE` log lines that identify each failing test, then add those classes to `<excludedTestClasses>` as shown in error E4.

35 E7 — Many `TIMED_OUT` Mutants in Results

E7 — Symptom

```
>> Generated N mutations Killed M (X%)
   (a large number of TIMED_OUT entries listed under multiple mutators)
```

Cause. `TIMED_OUT` results have two possible sources:

1. **Tests are inherently slow.** Some test methods take several seconds each. PIT's default `timeoutConstant` (4000 ms) is insufficient; the mutant's test run exceeds the deadline before any assertion fires.
2. **`timeoutConstant` is too low for the project.** Even moderately fast tests can time out if the forked minion JVM takes longer to start up on slower hardware.

In either case, PIT marks those mutants as killed-by-timeout rather than killed-by-assertion. This **inflates** the apparent mutation score and makes results unreliable.

Fix. Increase `timeoutConstant`. Start at 8000 ms; raise to 10000 ms if timeouts persist.

```
<!-- Raise from default 4000 to 8000 or 10000 if timeouts persist -->
<timeoutConstant>8000</timeoutConstant>
```

Listing 26: E7 fix: increase the per-mutant timeout

Note

Do not raise `timeoutConstant` beyond 10000 ms without also investigating *why* tests are slow. Genuinely infinite-loop mutants will always time out regardless of the value; only slow-but-finite tests benefit from a higher constant.

36 E8 — `BUILD SUCCESS` but Zero Mutations Generated

E8 — Symptom

```
>> Generated 0 mutations
[INFO] BUILD SUCCESS
```

Cause. The `targetClasses` glob pattern does not match any compiled class file. Either the package name is incorrect or the project has not been compiled before PIT ran.

Fix.

```
# 1. Recompile
mvn clean compile

# 2. Check what packages are actually compiled
ls target/classes/

# 3. Fix targetClasses to match exactly:
#    correct:   <param>com.google.gson.*</param>
#    too broad: <param>com.google.*</param>
```

Listing 27: E8 fix: verify and correct the targetClasses pattern

37 Quick-Reference Card

38 Essential Commands

```
# 0. Set Java 17
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH

# 1. Confirm Java version
mvn -version

# 2. Compile
mvn clean compile -q

# 3. Confirm green test suite (required before PIT)
mvn clean test

# 4. Run PIT and save the log
mvn org.pitest:pitest-maven:mutationCoverage 2>&1 | tee pit_run.log

# 5. Open the HTML report
xdg-open target/pit-reports/index.html        # Linux
open target/pit-reports/index.html            # macOS

# 6. List surviving mutants from the XML report
grep -A5 'status="SURVIVED"' target/pit-reports/mutations.xml | head -60
```

Listing 28: Complete quick-reference command list

39 Pre-Flight Checklist

Complete every item before running PIT.

- Project builds: `mvn clean install -DskipTests` passes
- `pom.xml` exists in working directory (Maven project confirmed)
- Test framework identified: JUnit 4, JUnit 5, or mixed
- Project structure confirmed: single-module or correct module selected
- Java 17+ confirmed (`mvn -version`)
- `mvn clean test` passes with 0 failures and 0 errors
- `pitest-maven` version is `1.15.3`
- `pitest-junit5-plugin:1.2.2` included **only if** project uses JUnit 5
- `pitest-junit5-plugin` is inside the plugin block only — not in top-level `<dependencies>`
- `targetClasses` matches the project's actual base package (Section 10)
- `targetTests` matches the same base package

`verbose` is set to `true`

Network or integration tests excluded if they trigger errors E4 or E6

Both `HTML` and `XML` are specified in `outputFormats`

Early checkpoint completed within first 2–3 days (Section 27)

40 Error Quick-Lookup

ID	Key symptom	One-line fix
E1	0 tests discovered	Set `JAVA_HOME` to Java 17
E2	0 tests; Java 17 already active	Move `junit5-plugin` inside the plugin block
E3	`OutputDirectoryCreator` not available	Upgrade `pitest-junit5-plugin` to `1.2.2`
E4	`UNKNOWN_ERROR` + `UnknownHostException`	Add network tests to `excludedTestClasses`
E5	`ClassNotFoundException` in minion	Add multi-release test to `excludedTestClasses`
E6	*N* tests did not pass without mutation	Enable verbose; find `SEVERE` lines; exclude those tests
E7	Many `TIMED_OUT` mutants	Tests slow or timeout too low; raise `timeoutConstant` to 8000–10000
E8	0 mutations generated	Fix `targetClasses` package pattern

Table 7: Error quick-lookup table